

Auth, routing & a live screen

Firestore Auth, go_router guards, and a WebSocket screen

Dinu-Ștefan Rusu

FILS, Universitatea Politehnica București · Week 12

What this lab is for



Gate the app on identity

Wire Firebase Auth (email and password, plus Google) so `authStateChanges` drives what the user sees.

TIME

2 hours



Guard routes, show live data

Add `go_router` with a redirect guard, then a screen that updates from a `WebSocket` without a refresh.

SUBMIT

Branch `lab-6`, `admd-labs` repo, before next lab

Before you write a line

1. Branch from the Lab 5 state: `git checkout -b lab-6`
2. Add packages: `firebase_auth`, `google_sign_in`, `go_router`, `web_socket_channel`.
3. Run `flutterfire configure` again to regenerate `firebase_options.dart`.
4. Register your Android debug *and* release SHA-1 fingerprints in the Firebase console. Google sign-in fails silently without them.

Hint

```
keytool -list -v -keystore ~/.android/debug.keystore -alias androiddebugkey
```

Sign-up and sign-in with Firebase Auth

```
Future<void> signUp(String email, String
    pw) async {
  try {
    await FirebaseAuth.instance
      .createUserWithEmailAndPassword(
        email: email.trim(), password:
        pw);
  } on FirebaseAuthException catch (e) {
    throw mapAuthError(e.code);
  }
}
```

1. Email, password, a sign-up/sign-in toggle, and `mapAuthError` for one message per code.

One generic message for a failed sign-in. Do not reveal whether an email is registered.

Google sign-in and the auth gate

```
Future<UserCredential> signInWithGoogle()
    async {
        final account =
            await
                GoogleSignIn.instance.authenticate();
        final token =
            account.authentication.idToken;
        final cred =
            GoogleAuthProvider.credential(
                idToken: token);
        return FirebaseAuth.instance
            .signInWithCredential(cred);
    }
```

DONE WHEN

- ☐ Google sign-in returns a signed-in user on a real device.
- ☐ Sign-out clears both sessions.

Call `initialize()` once at app start, before the first sign-in.

authStateChanges drives the app

```
StreamBuilder<User?>(
  stream: FirebaseAuth.instance.authStateChanges(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const SplashScreen();
    }
    final user = snapshot.data;
    return user == null ? const LoginScreen() : const HomeScreen();
  },
)
```

Listen to the stream. Never cache currentUser. A cached field goes stale after a sign-out elsewhere or a token refresh. This stream is what Task II hooks the router into.

go_router with a redirect guard

```
final router = GoRouter(  
  refreshListenable: GoRouterRefreshStream(  
    FirebaseAuth.instance.authStateChanges()),  
  redirect: (context, state) {  
    final signedIn = FirebaseAuth.instance  
      .currentUser != null;  
    final atLogin =  
      state.matchedLocation == '/login';  
    if (!signedIn && !atLogin) return '/login';  
    if (signedIn && atLogin) return '/';  
    return null; },  
); // routes: [...]
```

1. Add `/login`, `/`, `/live` routes.
2. Wire `refreshListenable` to `authStateChanges`.
3. Sign-out button on Home calls `signOut()`.

Why the guard belongs to the router



Without `refreshListenable`

The redirect only runs on navigation. A background sign-out leaves the private screen on display until the user taps something.



With it

Every auth change re-runs `redirect` immediately, including the very first frame and any deep link.

DONE WHEN

- ☐ Signing out from Home returns to `/login` without a manual navigation call.
- ☐ Opening `admd://live` while signed out lands on `/login`, not on `LiveScreen`.
- ☐ Signing in from `/login` lands back on `/`, never on `/login` itself.

One screen, live messages

```
final channel = WebSocketChannel.connect(
    Uri.parse('ws://10.0.2.2:8080/echo'));
await channel.ready;
StreamBuilder(
    stream: channel.stream,
    builder: (context, snapshot) => Text(
        snapshot.data as String? ?? 'Waiting...'),
)
```

await channel.ready is the line everyone forgets. `connect()` returns instantly and never throws. Without `ready`, a rejected handshake surfaces later as a stream error.

Building the live screen

1. Connect in `initState`, close in `dispose`.
2. Keep the last N messages in a list; show them in a `ListView`.
3. A text field and a send button that write to `channel.sink`.
4. Show a connection-state banner: connecting, live, disconnected.

DONE WHEN

- ☐ A message typed on one run appears without pressing refresh.
- ☐ Leaving the screen closes the socket (check the server log).
- ☐ The screen recovers after toggling airplane mode.

Hint

Android emulators reach your laptop's `localhost` at `10.0.2.2`, not `127.0.0.1`.

The echo server, in full

```
var upgrader websocket.Upgrader
func main() {
    http.HandleFunc("/echo", func(w http.ResponseWriter, r *http.Request) {
        conn, _ := upgrader.Upgrade(w, r, nil)
        defer conn.Close()
        for {
            mt, msg, err := conn.ReadMessage()
            if err != nil { return }
            conn.WriteMessage(mt, msg)
        }
    })
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Why a local echo server



No public dependency

A public echo endpoint can be slow, rate-limited, or offline during grading. This one runs on your laptop, on port 8080.

```
go mod init echo
go get github.com/gorilla/websocket
go run .
```



You can see both sides

The terminal running `go run .` prints every frame, which makes the WebSocket lifecycle from Lecture 11 concrete instead of abstract.

Errors and their fixes

`PlatformException(sign_in_failed,
ApiException: 10)`

The SHA-1 fingerprint is missing or wrong in the Firebase console. Re-run `keytool` and re-download `google-services.json`.

Too many redirects, or a blank screen at startup

`redirect` returned the location it is already on. Return `null` to allow, never the current route.

`WebSocketChannelException:
Connection refused`

The Go server is not running, or the emulator is using `127.0.0.1` instead of `10.0.2.2`.

`setState() called after dispose()`

A stream event arrived after the screen closed. Cancel the subscription in `dispose` before calling `sink.close`.

Push before you leave.

Branch lab-6 · one commit per task · echo server included in the repo